

# Database 1 Project

# Modeling and Implementation

*Instructor: Varazdat Avetisyan*

# Content

|                                                   |    |
|---------------------------------------------------|----|
| 1. Introduction .....                             | 2  |
| 1.1 Problem Description .....                     | 3  |
| Main Stakeholders and Their Goals .....           | 3  |
| 1.2 System Requirements .....                     | 4  |
| 2. Conceptual Design .....                        | 7  |
| 2.1 Main Data Objects and Business Rules .....    | 8  |
| Main Data Objects .....                           | 8  |
| Business Rules .....                              | 8  |
| 2.2 System Data Requirements and Processes .....  | 9  |
| Data Requirements .....                           | 9  |
| System Processes .....                            | 9  |
| 2.3 System Diagrams .....                         | 10 |
| 3. Logical Design .....                           | 14 |
| 3.1 Entity–Relationship (ER) Diagram .....        | 15 |
| 3.2 Relational Schema .....                       | 17 |
| 3.3 Normalization Process .....                   | 18 |
| 3.3.1 Approach .....                              | 19 |
| First Normal Form (1NF) .....                     | 19 |
| Second Normal Form (2NF) .....                    | 19 |
| Third Normal Form (3NF) .....                     | 19 |
| Conclusion .....                                  | 20 |
| 3.4 Relational Algebra Query Demonstrations ..... | 20 |
| Operations Used .....                             | 21 |
| Example Queries .....                             | 21 |
| 4. Physical Implementation .....                  | 26 |
| 4.1 DDL – Data Definition Language .....          | 27 |
| 4.2 DML – Data Manipulation Language .....        | 28 |
| 4.3 DQL – Data Query Language .....               | 29 |
| 4.4 Views .....                                   | 29 |
| 4.5 Indexes .....                                 | 30 |
| 4.6 Triggers .....                                | 32 |

|                                         |    |
|-----------------------------------------|----|
| 4.7 Stored Procedures .....             | 33 |
| 4.8 DCL – Data Control Language.....    | 34 |
| 5. Database Deployment .....            | 35 |
| 5.1 Deployment Stack.....               | 36 |
| 5.2 Docker Compose Architecture .....   | 36 |
| 5.3 AWS Security Configuration .....    | 37 |
| 6. Project Conclusions and Results..... | 37 |
| 6.1 Summary of Deliverables .....       | 38 |
| 6.2 Key Design Decisions .....          | 38 |
| 6.3 Possible Future Improvements.....   | 39 |
| Conclusion .....                        | 39 |

# 1. INTRODUCTION

## 1.1 PROBLEM DESCRIPTION

The selected problem domain is a **Coding Challenge and Pixel Ownership Platform**, a gamified system that integrates algorithmic problem-solving with a shared pixel-grid environment. The platform allows users to register, participate in coding challenges, and interact within a competitive and dynamic ecosystem.

Each challenge is associated with a set of test cases used to automatically evaluate user submissions. When a user submits a solution, the system verifies its correctness against all test cases and records detailed information such as execution time and result status. For more precise evaluation, each submission can also store individual results per test case.

Upon successful completion of a challenge, the system grants the user ownership of one or more pixels linked to that challenge, creating a visual and interactive representation of achievements.

The platform also maintains a leaderboard, which aggregates user performance across all challenges, encouraging competition and continuous engagement. Additionally, every change in pixel ownership is recorded in a dedicated history log, ensuring full traceability and supporting auditing or replay of system events.

The primary objective of the system is to efficiently manage and organize data related to users, administrators, challenges, test cases, submissions, pixel ownership, and rankings, while ensuring consistency, integrity, and adherence to defined business rules.

## Main Stakeholders and Their Goals

**Players (Users):** Aim to solve coding challenges, submit solutions, and gain pixel ownership while improving their ranking on the leaderboard.

**Challenge Creators (Admins/Moderators):** Responsible for creating and managing coding challenges, defining test cases, and ensuring the quality and fairness of problems.

## 1.2 SYSTEM REQUIREMENTS

### 1.2.1 Functional Requirements

These define the main operations supported by the system:

#### **User Management**

- Users must be able to register, log in, and manage their profiles
- The system must store user credentials securely and enforce unique usernames and emails

#### **Challenge Management**

- Administrators must be able to create, update, and manage coding challenges
- Each challenge must include a difficulty level and associated test cases

#### **Submission Handling**

- Users must be able to submit solutions for challenges
- The system must evaluate submissions automatically using test cases
- Each submission must store result status and execution time
- The system must store detailed results for each test case per submission

#### **Pixel Ownership System**

- Each pixel must be associated with a specific challenge
- Successful submissions must trigger automatic pixel ownership assignment or transfer
- The system must maintain the current owner of each pixel
- Each pixel must have a unique position in the grid

#### **History Tracking**

- All changes in pixel ownership must be recorded in a history table
- The system must allow tracking and reviewing past ownership states

### **Leaderboard System**

- The system must calculate user scores dynamically based on performance
- Users must be ranked based on their performance

### **Submission Control**

- The system must prevent rapid duplicate submissions for the same challenge within a defined time interval

## **1.2.2 Non-Functional Requirements**

These define system quality and constraints:

### **Data Integrity**

- The system must enforce consistency using primary keys, foreign keys, and constraints

### **Performance**

- The system must efficiently handle frequent submissions and evaluation processes

### **Security**

- The system must ensure secure handling of user data and prevent unauthorized access

## **Access Control**

- The system must implement role-based access with separate administrator and user roles



## 2. CONCEPTUAL DESIGN

### 2.1 MAIN DATA OBJECTS AND BUSINESS RULES

#### Main Data Objects

- **Users:** Stores user account information, including credentials and registration data
- **Admins:** Stores administrator credentials responsible for managing the platform
- **Challenges:** Represents coding problems with descriptions, difficulty levels, and creators
- **TestCases:** Contains input and expected output used for automatic validation of solutions
- **Submissions:** Stores user-submitted solutions along with execution time and result status
- **SubmissionTestResults:** Stores detailed per-test-case evaluation results for each submission
- **Pixels:** Represents elements of the grid that can be owned by users
- **PixelHistory:** Maintains a record of all ownership changes for each pixel
- **Leaderboard:** Stores user scores and ranking positions

#### Business Rules

- Each user must have a unique identifier and email
- Each challenge must be created by a valid administrator
- A challenge can have multiple test cases
- A user can submit multiple solutions for a challenge
- Each submission must be evaluated and assigned a result
- Each submission must include detailed results for each test case
- A pixel must be associated with exactly one challenge

- A pixel can have only one owner at any given time
- Each pixel must have a unique coordinate position
- Pixel ownership is assigned or transferred only after a successful submission
- All ownership changes must be recorded in the PixelHistory table
- User rankings must be derived from submission performance
- The system must prevent multiple submissions from the same user for the same challenge within a short time interval

## 2.2 SYSTEM DATA REQUIREMENTS AND PROCESSES

### Data Requirements

The system must store and manage the following types of data:

- User data: identifiers, usernames, emails, passwords, timestamps
- Admin data: credentials for system management
- Challenge data: titles, descriptions, difficulty levels, creators
- Test case data: inputs and expected outputs
- Submission data: code, execution time, results, timestamps
- Detailed evaluation data: per-test-case results for each submission
- Pixel data: coordinates, owner, associated challenge
- Historical data: previous and new pixel ownership records
- Leaderboard data: user scores and rankings

### System Processes

The system operates through the following main processes:

- **User Registration and Authentication:**  
Users create accounts and log into the system

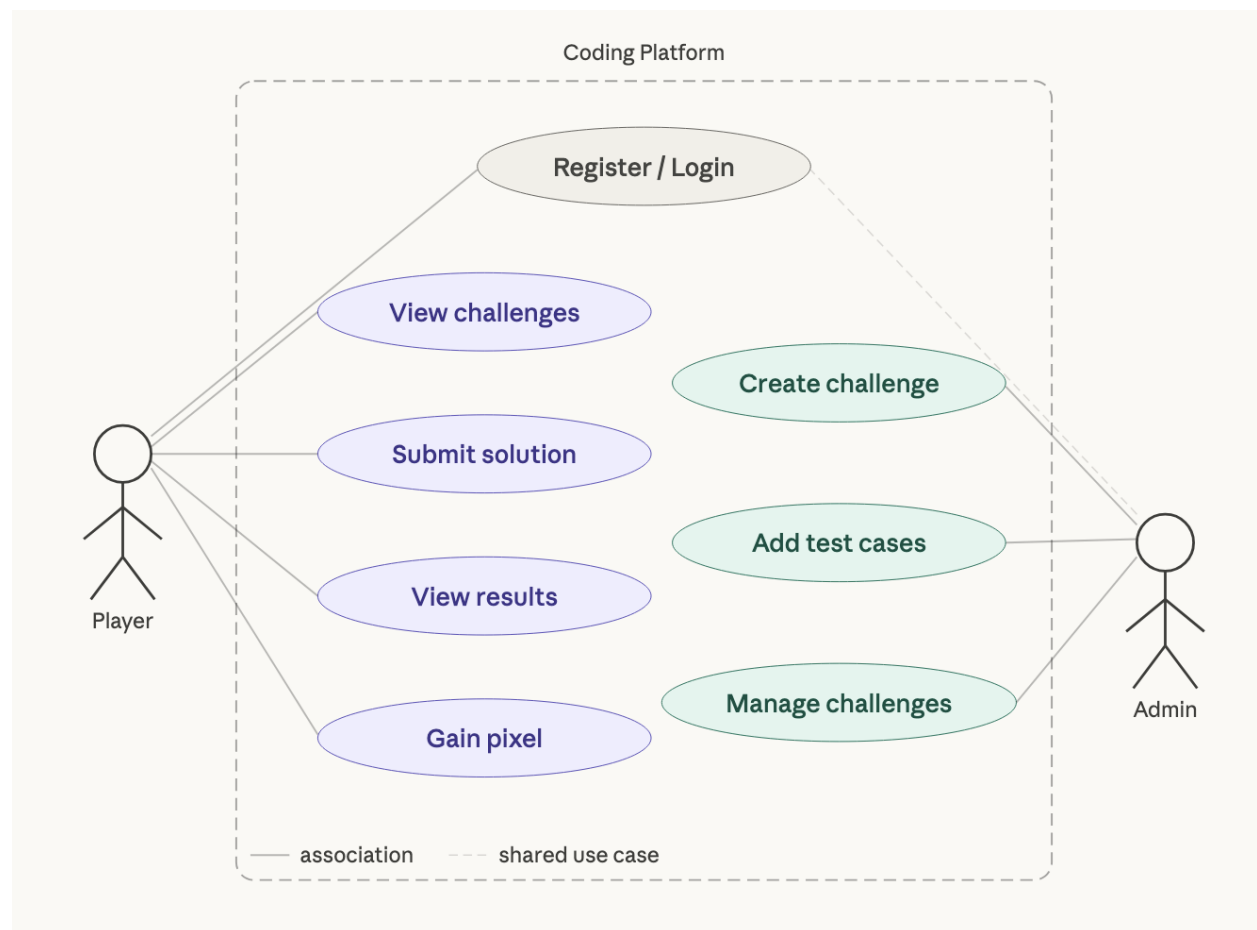
- **Challenge** **Creation** **and** **Management:**  
Administrators define and manage coding challenges and their test cases
- **Solution** **Submission** **and** **Evaluation:**  
Users submit code solutions, which are automatically validated
- **Pixel** **Ownership** **Management:**  
Successful submissions trigger assignment or transfer of pixel ownership
- **Leaderboard** **Update:**  
User scores are updated based on successful challenge completions
- **History** **Logging:**  
All changes in pixel ownership are recorded for tracking and auditing

## 2.3 SYSTEM DIAGRAMS

To visually represent the system structure and behavior, the following diagrams are used:

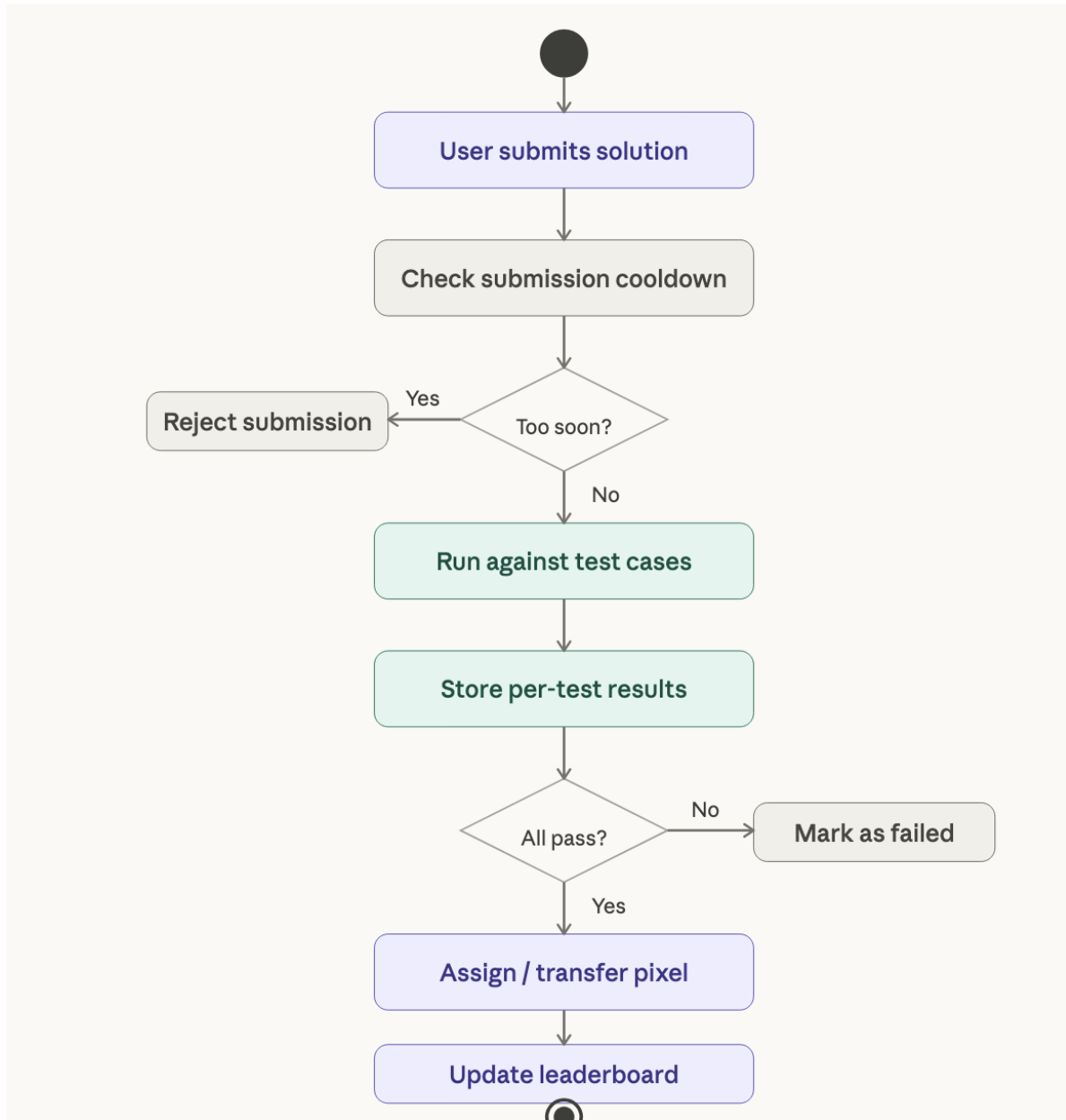
- **Use Case Diagram:**

Illustrates the interactions between users and administrators with the system. It shows the main functionalities available to each role, such as submitting solutions for users and managing challenges for administrators.



- **Activity Diagram:**

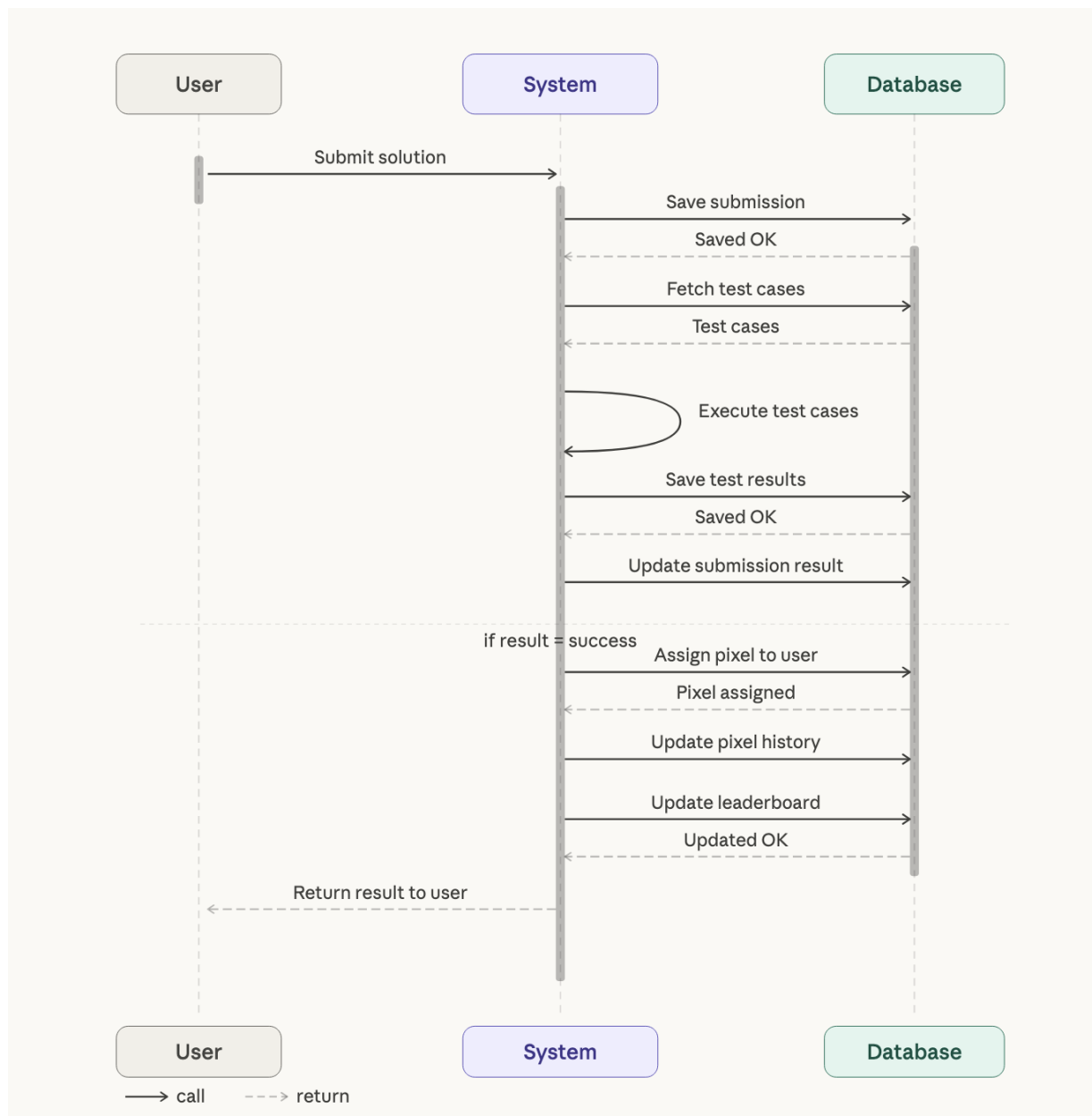
The Activity Diagram represents the workflow of solution submission and evaluation. It shows how the system processes submissions, checks test cases, and updates pixel ownership based on the result.



- Sequence

Diagram:

Shows the order of interactions between the user, system, and database during a submission. It highlights how the system processes the request, evaluates the solution, and returns the result.





## 3. LOGICAL DESIGN

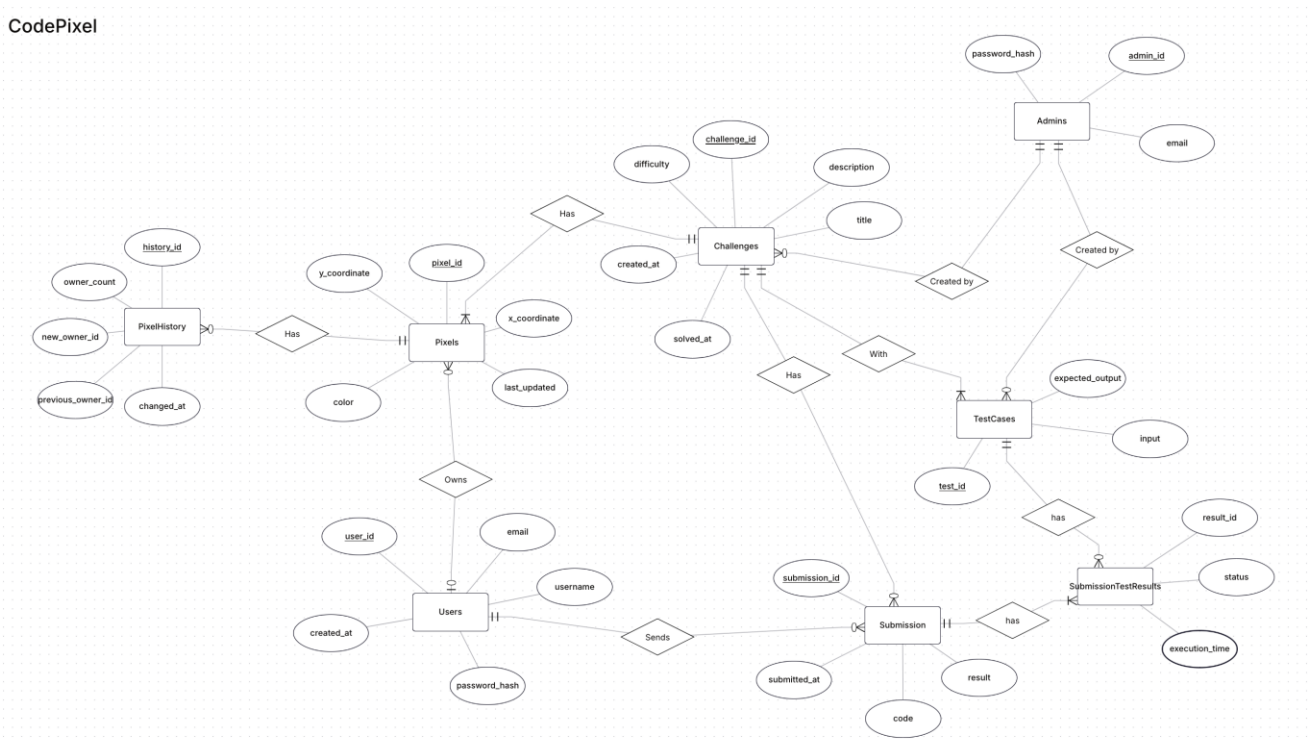
### 3.1 ENTITY-RELATIONSHIP (ER) DIAGRAM

The Entity-Relationship (ER) diagram represents the conceptual structure of the system, illustrating all entities, their attributes, and the relationships between them. The diagram follows standard ER modeling conventions, where entities are represented as rectangles, attributes as ellipses (with primary keys underlined), and relationships as diamonds. The system consists of the following main entities:

- Users
- Admins
- Challenges
- TestCases
- Submissions
- SubmissionTestResults
- Pixels
- PixelHistory



## CodePixel



The ER diagram defines the following relationships and cardinalities:

### 1. **Users — Submits — Submissions (1:N):**

One user can submit multiple solutions

### 2. **Challenges — Receives — Submissions (1:N):**

Each challenge can receive multiple submissions

### 3. **Admins — Creates — Challenges (1:N):**

One administrator can create multiple challenges

4. **Challenges — Contains — TestCases (1:N):**

Each challenge is associated with multiple test cases

5. **Submissions — Has — SubmissionTestResults (1:N):**

Each submission contains multiple test case results

6. **TestCases — Evaluates — SubmissionTestResults (1:N):**

Each test case can be linked to multiple submission results

7. **Users — Owns — Pixels (1:N):**

A user can own multiple pixels, while each pixel has at most one owner

8. **Challenges — Assigns — Pixels (1:N):**

Each pixel is associated with a specific challenge

9. **Pixels — Records — PixelHistory (1:N):**

Each pixel maintains a history of ownership changes

10. **PixelHistory — References — Users (N:1):**

Each history record stores previous and new owners

All relationships are defined with appropriate cardinalities and constraints to ensure data consistency and reflect real-world system behavior.

## 3.2 RELATIONAL SCHEMA

The ER model is transformed into a relational schema by mapping each entity to a table and each relationship to foreign keys. The resulting schema consists of seven normalized tables.

| Table                      | Primary Key   | Foreign Keys                                                             | Constraints                           |
|----------------------------|---------------|--------------------------------------------------------------------------|---------------------------------------|
| USERS                      | user_id       | —                                                                        | UNIQUE(username),<br>UNIQUE(email)    |
| ADMINS                     | admin_id      | —                                                                        | UNIQUE(email)                         |
| CHALLENGES                 | challenge_id  | admin_id → ADMINS                                                        | CHECK(difficulty<br>BETWEEN 1 AND 5)  |
| TESTCASES                  | test_id       | challenge_id → CHALLENGES                                                | —                                     |
| SUBMISSIONS                | submission_id | user_id → USERS, challenge_id<br>→ CHALLENGES                            | —                                     |
| SUBMISSIONS<br>TESTRESULTS | result_id     | submission_id →<br>SUBMISSIONS, test_id →<br>TESTCASES                   | —                                     |
| PIXELS                     | pixel_id      | owner_id → USERS,<br>challenge_id → CHALLENGES                           | UNIQUE(x_coordinate,<br>y_coordinate) |
| PIXELHISTORY               | history_id    | pixel_id → PIXELS,<br>new_owner_id → USERS,<br>previous_owner_id → USERS | —                                     |

Each table is defined with a primary key to uniquely identify records and foreign keys to maintain referential integrity between related entities.

## 3.3 NORMALIZATION PROCESS

### 3.3.1 Approach

Normalization was applied during the design phase by modeling each entity and relationship as a separate table. This approach minimizes redundancy, prevents anomalies, and ensures clear data dependencies.

#### First Normal Form (1NF)

All tables satisfy 1NF as:

- All attributes are atomic
- No repeating groups or multi-valued attributes exist

For example:

- Submissions are stored as separate rows instead of being embedded within Users or Challenges
- Test cases and submission results are stored individually

#### Second Normal Form (2NF)

All tables satisfy 2NF because:

- Each table has a single-attribute primary key
- All non-key attributes fully depend on the primary key

For example:

- In SUBMISSIONS, all attributes depend on submission\_id
- SubmissionTestResults depends fully on result\_id

### **Third Normal Form (3NF)**

All tables satisfy 3NF as:

- No transitive dependencies exist
- Non-key attributes depend only on the primary key

Key design decisions:

- User information is stored only in USERS
- PIXELS stores only user\_id, not user details
- Pixel ownership history is separated into PIXELHISTORY
- Submission evaluation details are separated into SUBMISSIONTESTRESULTS
- This ensures consistency and eliminates update anomalies.

This ensures consistency and eliminates update anomalies.

### **Conclusion**

The schema satisfies 1NF, 2NF, and 3NF. The normalized structure ensures efficient data storage, eliminates redundancy, and maintains referential integrity.

## 3.4 RELATIONAL ALGEBRA QUERY DEMONSTRATIONS

A set of relational algebra queries is used to demonstrate the database's ability to support various data retrieval operations. These queries reflect real-world use cases of system.

### Operations Used

The following relational algebra operations are applied:

- **Selection ( $\sigma$ ):** Filtering rows based on conditions
- **Projection ( $\pi$ ):** Selecting specific attributes
- **Join ( $\bowtie$ ):** Combining related tables
- **Union ( $\cup$ ):** Combining results from multiple relations
- **Intersection ( $\cap$ ):** Finding common records
- **Difference ( $-$ ):** Identifying non-overlapping records
- **Rename ( $\rho$ ):** Renaming attributes
- **Grouping ( $\gamma$ ):** Aggregation operations

### Example Queries

**1. Retrieve all users** Returns the usernames of all registered users in the system.  $\pi$  username (USERS)

**SQL:**

```
SELECT username FROM USERS;
```

**2. Retrieve all submissions of a specific user** Filters submissions to show only those created by a particular user.  $\sigma$  user\_id=1 (SUBMISSIONS)

**SQL:**

```
SELECT * FROM SUBMISSIONS WHERE user_id = 1;
```

**3. Retrieve accepted submissions** Selects only submissions that have a successful result status.  $\sigma$  result='Accepted' (SUBMISSIONS)

**SQL:**

```
SELECT * FROM SUBMISSIONS WHERE result = 'Accepted';
```

**4. Retrieve all challenges** Displays the titles of all available coding challenges.  $\pi$  title (CHALLENGES)

**SQL:**

```
SELECT title FROM CHALLENGES;
```

**5. Retrieve test cases of a specific challenge** Returns all test cases associated with a given challenge.  $\sigma$  challenge\_id=1 (TESTCASES)

**SQL:**

```
SELECT * FROM TESTCASES WHERE challenge_id = 1;
```

**6. Retrieve submissions with user information** Combines user and submission data to show which user made each submission.  $\pi$  username, submission\_id (USERS  $\bowtie$  SUBMISSIONS)

**SQL:**

```
SELECT u.username, s.submission_id  
FROM USERS u  
JOIN SUBMISSIONS s ON u.user_id = s.user_id;
```





**7. Retrieve submissions for a specific challenge** Filters submissions that belong to a particular challenge.  $\sigma$  challenge\_id=1 (SUBMISSIONS)

**SQL:**

```
SELECT * FROM SUBMISSIONS WHERE challenge_id = 1;
```

**8. Retrieve pixels owned by a specific user** Returns all pixels currently owned by a given user.  $\sigma$  user\_id=1 (PIXELS)

**SQL:**

```
SELECT * FROM PIXELS WHERE user_id = 1;
```

**9. Retrieve pixel ownership history** Displays pixel ownership changes including the new owner of each pixel.  $\pi$  pixel\_id, new\_owner\_id (PIXELHISTORY)

**SQL:**

```
SELECT pixel_id, new_owner_id FROM PIXELHISTORY;
```

**10. Retrieve submission test results** Returns the status of test case evaluations for submissions.  $\pi$  status (SUBMISSIONTESTRESULTS)

**SQL:**

```
SELECT status FROM SUBMISSIONTESTRESULTS;
```

**11. Retrieve failed test case results** Filters only the test results where execution failed.  $\sigma$  status='Failed' (SUBMISSIONTESTRESULTS)

**SQL:**

```
SELECT * FROM SUBMISSIONTESTRESULTS WHERE status = 'Failed';
```

**12. Retrieve users who have submitted solutions** Finds users who have at least one submission using a join operation.  $\pi \text{ username } (\text{USERS} \bowtie \text{SUBMISSIONS})$

**SQL:**

```
SELECT DISTINCT u.username
FROM USERS u
JOIN SUBMISSIONS s ON u.user_id = s.user_id;
```

**13. Retrieve challenges that have test cases** Returns challenges that are properly configured with test cases.  $\pi \text{ title } (\text{CHALLENGES} \bowtie \text{TESTCASES})$

**SQL:**

```
SELECT DISTINCT c.title
FROM CHALLENGES c
JOIN TESTCASES t ON c.challenge_id = t.challenge_id;
```

**14. Retrieve pixels without an owner** Identifies pixels that are not currently assigned to any user.  $\sigma \text{ user\_id IS NULL } (\text{PIXELS})$

**SQL:**

```
SELECT * FROM PIXELS WHERE user_id IS NULL;
```

**15. Retrieve users who have not submitted any solutions** Finds users who do not appear in the submissions table using set difference.  $\pi \text{ user\_id } (\text{USERS}) - \pi \text{ user\_id } (\text{SUBMISSIONS})$

**SQL:**

```
SELECT user_id
FROM USERS
WHERE user_id NOT IN (
    SELECT user_id FROM SUBMISSIONS
);
```

**16. Calculate total successful submissions per user** Counts how many challenges each user has successfully solved.  $\gamma$  user\_id, COUNT(submission\_id)  $\rightarrow$  total\_success ( $\sigma$  result='Accepted' (SUBMISSIONS))

**SQL:**

```
SELECT user_id, COUNT(submission_id) AS total_success
FROM SUBMISSIONS
WHERE result = 'Accepted'
GROUP BY user_id;
```

**17. Calculate number of submissions per challenge** Counts how many times each challenge has been attempted by users.  $\gamma$  challenge\_id, COUNT(submission\_id)  $\rightarrow$  total\_attempts (SUBMISSIONS)

**SQL:**

```
SELECT challenge_id, COUNT(submission_id) AS total_attempts
FROM SUBMISSIONS
GROUP BY challenge_id;
```

**18. Calculate leaderboard ranking (user scores)** Aggregates successful submissions per user to represent their score.  $\gamma$  user\_id, COUNT(submission\_id)  $\rightarrow$  score ( $\sigma$  result='Accepted' (SUBMISSIONS))

**SQL:**

```
SELECT user_id, COUNT(submission_id) AS score
FROM SUBMISSIONS
WHERE result = 'Accepted'
GROUP BY user_id
ORDER BY score DESC;
```

**Conclusion:** These queries demonstrate the flexibility of the database design and its ability to support essential operations such as filtering, joining, and data analysis. They reflect common real-world use cases of the system and validate the completeness of the relational model.

## 4. PHYSICAL IMPLEMENTATION

This section presents the complete SQL implementation of the Coding Challenge and Pixel Ownership Platform. All database objects were created and tested using MySQL 8.0, deployed inside a Docker container and hosted on an AWS EC2 instance.

### 4.1 DDL – DATA DEFINITION LANGUAGE

The schema is defined in 01\_schema.sql. It creates all tables with primary keys, foreign keys, constraints, and ENUM types.

#### Database and core tables:

```
DROP DATABASE IF EXISTS coding_challenge_platform;
CREATE DATABASE coding_challenge_platform;
USE coding_challenge_platform;
```

#### Key design decisions:

- ENUM types enforce valid values for difficulty ('easy','medium','hard'), result ('Accepted','Wrong Answer','Time Limit Exceeded','Runtime Error'), and language ('python','java','cpp','javascript','sql').
- All foreign keys use ON UPDATE CASCADE to propagate key changes.
- Pixels use a composite UNIQUE constraint on (challenge\_id, x\_coordinate, y\_coordinate) to guarantee a unique grid position per challenge.
- A CHECK constraint on pixel color enforces valid hex color format (#RRGGBB).
- The pixel\_history table tracks all ownership and color changes with a change\_type ENUM ('ASSIGNED','REASSIGNED','RELEASED','RECOLORED').

#### Tables created:

| Table      | Primary Key  | Key Constraints                              |
|------------|--------------|----------------------------------------------|
| admins     | admin_id     | UNIQUE(username), UNIQUE(email)              |
| users      | user_id      | UNIQUE(username), UNIQUE(email)              |
| challenges | challenge_id | FK(created_by_admin_id),<br>ENUM(difficulty) |
| test_cases | test_case_id | FK(challenge_id),<br>FK(created_by_admin_id) |

|                         |                           |                                                                                     |
|-------------------------|---------------------------|-------------------------------------------------------------------------------------|
| submissions             | submission_id             | FK(user_id), FK(challenge_id),<br>ENUM(result, language)                            |
| pixels                  | pixel_id                  | FK(challenge_id), FK(owner_user_id),<br>UNIQUE(challenge_id, x, y),<br>CHECK(color) |
| pixel_history           | history_id                | FK(pixel_id), FK(challenge_id),<br>FK(prev/new owner)                               |
| submission_test_results | submission_test_result_id | FK(submission_id), FK(test_case_id),<br>UNIQUE(submission_id, test_case_id)         |

## 4.2 DML – DATA MANIPULATION LANGUAGE

All data insertion is handled in 02\_data.sql using WITH RECURSIVE sequences. This approach generates large, consistent datasets compactly without long repetitive INSERT blocks.

| Table                   | Records Inserted | Generation Method                           |
|-------------------------|------------------|---------------------------------------------|
| admins                  | 45               | WITH RECURSIVE seq (n = 1..45)              |
| users                   | 50               | WITH RECURSIVE seq (n = 1..50)              |
| challenges              | 45               | WITH RECURSIVE seq (n = 1..45)              |
| test_cases              | 90               | 2 per challenge                             |
| pixels                  | 90               | 2 per challenge, initially unclaimed        |
| submissions             | 120              | Spaced 1 minute apart to avoid spam trigger |
| submission_test_results | 120              | Derived from submissions table              |

Example — recursive user insert:

```
INSERT INTO users (username, email, password_hash)
WITH RECURSIVE seq AS (
    SELECT 1 AS n UNION ALL SELECT n + 1 FROM seq WHERE n < 50
)
SELECT CONCAT('user', LPAD(n,2,'0')),
       CONCAT('user', LPAD(n,2,'0'), '@pixelplatform.com'),
       CONCAT('user_hash_', LPAD(n,2,'0'))
FROM seq;
```

Submission timestamps are spaced one minute apart (TIMESTAMPADD(MINUTE, n, ...)) to avoid triggering the anti-spam trigger during seeding.

### 4.3 DQL – DATA QUERY LANGUAGE

All queries are stored in 03\_queries.sql. The following table documents each query, its purpose, and the SQL used.

| #  | Query Purpose                       | SQL Summary                                                                                    |
|----|-------------------------------------|------------------------------------------------------------------------------------------------|
| 1  | All users ordered by ID             | SELECT user_id, username, email, created_at<br>FROM users ORDER BY user_id                     |
| 2  | All challenges with difficulty      | SELECT challenge_id, title, difficulty, reward_pixels<br>FROM challenges ORDER BY challenge_id |
| 3  | Full submission details (JOIN)      | SELECT s.*, u.username, c.title FROM submissions<br>s JOIN users u JOIN challenges c           |
| 4  | Accepted submissions per challenge  | GROUP BY challenge_id WHERE result='Accepted'<br>ORDER BY count DESC                           |
| 5  | Submission count per user           | GROUP BY user_id, username ORDER BY<br>total_submissions DESC                                  |
| 6  | Pixel ownership count per user      | JOIN pixels ON owner_user_id, GROUP BY user_id                                                 |
| 7  | Pixel history ordered newest first  | SELECT FROM pixel_history ORDER BY<br>changed_at DESC                                          |
| 8  | Test results per submission         | JOIN submission_test_results ON submission_id,<br>ORDER BY submission_id                       |
| 9  | Fastest accepted time per challenge | MIN(execution_time_ms) WHERE result='Accepted'<br>GROUP BY challenge_id                        |
| 10 | Users with no submissions           | LEFT JOIN submissions WHERE submission_id IS<br>NULL                                           |

## 4.4 VIEWS

Six views are defined in 04\_views.sql and four additional web-facing views in 10\_web\_views.sql. Views provide reusable, named queries that simplify application-layer data access.

| View Name                  | Purpose                                                   |
|----------------------------|-----------------------------------------------------------|
| vw_leaderboard             | Aggregates accepted submissions and owned pixels per user |
| vw_user_submission_history | Full submission details joined with user and challenge    |
| vw_challenge_performance   | Attempt counts and fastest accepted time per challenge    |
| vw_pixel_ownership         | All pixels with owner username and challenge title        |
| vw_pixel_history           | Ownership changes with resolved usernames                 |
| vw_submission_test_results | Test results joined with submission and challenge         |
| vw_web_leaderboard         | Leaderboard with RANK() window function for frontend      |
| vw_web_canvas              | Full pixel grid with owner info for PixelCanvas page      |
| vw_web_recent_activity     | Last 50 pixel ownership changes for live feed             |
| vw_web_challenge_list      | Challenge list with attempt counts for UI display         |

**Example — leaderboard view using subqueries to avoid count inflation:**

```
CREATE VIEW vw_leaderboard AS
SELECT u.user_id, u.username,
       COALESCE(s.accepted_submissions, 0) AS accepted_submissions,
       COALESCE(p.owned_pixels, 0) AS owned_pixels
FROM users AS u
LEFT JOIN (SELECT user_id,
                  SUM(CASE WHEN result='Accepted' THEN 1 ELSE 0 END) AS
accepted_submissions
          FROM submissions GROUP BY user_id) AS s ON s.user_id = u.user_id
LEFT JOIN (SELECT owner_user_id AS user_id, COUNT(*) AS owned_pixels
          FROM pixels WHERE owner_user_id IS NOT NULL
          GROUP BY owner_user_id) AS p ON p.user_id = u.user_id;
```

## 4.5 INDEXES

Ten indexes are defined in 05\_indexes.sql to optimize the most frequent query patterns used by views, triggers, and procedures.

| Index Name                   | Table         | Columns                     | Purpose                                        |
|------------------------------|---------------|-----------------------------|------------------------------------------------|
| idx_submissions_user_id      | submissions   | user_id                     | Speeds up per-user submission queries          |
| idx_submissions_challenge_id | submissions   | challenge_id                | Speeds up per-challenge queries                |
| idx_submissions_result_time  | submissions   | result, execution_time_ms   | Optimizes leaderboard and fastest-time queries |
| idx_test_cases_challenge_id  | test_cases    | challenge_id                | Speeds up test case lookup per challenge       |
| idx_pixels_owner_user_id     | pixels        | owner_user_id               | Optimizes pixel ownership queries              |
| idx_pixels_challenge_owner   | pixels        | challenge_id, owner_user_id | Composite index for trigger pixel assignment   |
| idx_pixel_history_pixel_id   | pixel_history | pixel_id                    | Speeds up history lookup per pixel             |



|                                           |                         |                   |                                       |
|-------------------------------------------|-------------------------|-------------------|---------------------------------------|
| idx_pixel_history_new_owner               | pixel_history           | new_owner_user_id | Optimizes new-owner history queries   |
| idx_submission_test_results_submission_id | submission_test_results | submission_id     | Speeds up test result joins           |
| idx_submission_test_results_test_case_id  | submission_test_results | test_case_id      | Speeds up per-test-case result lookup |

## 4.6 TRIGGERS

Four triggers are defined in 06\_triggers.sql to automate business rules without requiring application-layer logic.

| Trigger Name                              | Event                                    | Purpose                                                                                                                                |
|-------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| trg_submissions_before_insert             | BEFORE INSERT on submissions             | Validates execution_time_ms, memory_kb, submitted_at. Enforces 5-second minimum between submissions for the same user and challenge.   |
| trg_submissions_after_insert              | AFTER INSERT on submissions              | If result = 'Accepted', finds the first unclaimed pixel for that challenge and assigns it to the user.                                 |
| trg_pixels_after_update                   | AFTER UPDATE on pixels                   | Uses NULL-safe comparison (<=>) to detect owner or color changes and inserts a record into pixel_history with the correct change_type. |
| trg_submission_test_results_before_insert | BEFORE INSERT on submission_test_results | If execution_time_ms is missing, inherits the value from the parent submission row.                                                    |

Example — pixel assignment trigger:

```
CREATE TRIGGER trg_submissions_after_insert
AFTER INSERT ON submissions FOR EACH ROW
BEGIN
    DECLARE v_pixel_id INT DEFAULT NULL;
    IF NEW.result = 'Accepted' THEN
        SELECT pixel_id INTO v_pixel_id FROM pixels
        WHERE challenge_id = NEW.challenge_id AND owner_user_id IS NULL
        ORDER BY x_coordinate, y_coordinate LIMIT 1;
        IF v_pixel_id IS NOT NULL THEN
            UPDATE pixels SET owner_user_id = NEW.user_id,
                acquired_submission_id = NEW.submission_id
            WHERE pixel_id = v_pixel_id;
        END IF;
    END IF;
END$$
```

## 4.7 STORED PROCEDURES

Eleven stored procedures are defined across 07\_procedures.sql and 09\_auth\_procedures.sql, covering both system logic and frontend API needs.

| Procedure                | Parameters                                                                                | Purpose                                                               |
|--------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| sp_create_submission     | user_id, challenge_id, code, language, result, execution_time_ms, memory_kb, submitted_at | Main entry point for creating a submission from the application       |
| sp_get_user_score        | user_id                                                                                   | Returns accepted submission count and owned pixel count for a user    |
| sp_get_user_activity     | user_id                                                                                   | Returns all submissions for a user, ordered newest first              |
| sp_get_challenge_summary | challenge_id                                                                              | Returns attempt counts, fastest time, and pixel stats for a challenge |
| sp_reassign_pixel        | pixel_id, new_owner_user_id, submission_id, new_color                                     | Reassigns a pixel — triggers pixel_history log automatically          |
| sp_register              | username, email, password_hash                                                            | Registers a new user; raises error on duplicate username or email     |
| sp_login                 | username, password_hash                                                                   | Returns user row if credentials match; empty result if not            |
| sp_get_profile           | user_id                                                                                   | Returns user info + owned pixel list for the Profile page             |
| sp_get_leaderboard       | —                                                                                         | Returns top 50 users ranked by accepted submissions                   |
| sp_get_canvas            | challenge_id                                                                              | Returns full pixel grid with owner usernames for PixelCanvas          |
| sp_get_pixel_history     | limit                                                                                     | Returns recent pixel ownership changes for the activity feed          |

## 4.8 DCL – DATA CONTROL LANGUAGE

Access control is defined in 08\_dcl.sql using two database users with different privilege levels.

| DB User              | Privileges                      | Used By                                             |
|----------------------|---------------------------------|-----------------------------------------------------|
| app_user@localhost   | SELECT, INSERT, UPDATE, EXECUTE | Frontend backend API — read, write, call procedures |
| admin_user@localhost | ALL PRIVILEGES                  | Database administrator — full access                |

Note: DELETE is intentionally not granted to app\_user to prevent accidental or malicious data loss from the application layer. Pixel releases are handled through UPDATE (setting owner to NULL) rather than DELETE.

## 5. DATABASE DEPLOYMENT

The database is deployed using Docker and hosted on an AWS EC2 instance, making it accessible from both the frontend application and local development tools.

### 5.1 DEPLOYMENT STACK

| Component               | Technology                     | Purpose                               |
|-------------------------|--------------------------------|---------------------------------------|
| Database Engine         | MySQL 8.0 (Docker)             | Core relational database              |
| Container Orchestration | Docker Compose                 | Manages all services in one YAML file |
| Cloud Host              | AWS EC2 t3.small               | Ubuntu 22.04, 2 vCPU, 2 GB RAM        |
| Storage                 | AWS EBS gp3, 20 GB             | Persistent volume for MySQL data      |
| DB Admin GUI            | phpMyAdmin (port 8080)         | Browser-based database management     |
| Alt Admin GUI           | Adminer (port 8081)            | Lightweight alternative GUI           |
| Frontend                | React + Vite + nginx (port 80) | Served via Docker multi-stage build   |

### 5.2 DOCKER COMPOSE ARCHITECTURE

All services run inside a shared Docker network (pixelnet) using bridge mode. Container names are used as hostnames between services.

```
services:
  db:          mysql:8.0  → port 3306
  phpmyadmin:  phpmyadmin → port 8080  (depends on db)
  adminer:     adminer    → port 8081  (depends on db)
  frontend:    build ./frontend/my-app → port 80
```

SQL initialization files are mounted into /docker-entrypoint-initdb.d/ so MySQL runs them automatically on first boot in alphabetical order.

## 5.3 AWS SECURITY CONFIGURATION

The EC2 security group is configured with minimal open ports:

| Port | Service          | Source             |
|------|------------------|--------------------|
| 22   | SSH              | Developer IP only  |
| 80   | Frontend (nginx) | 0.0.0.0/0 (public) |
| 3306 | MySQL            | Developer IP only  |
| 8080 | phpMyAdmin       | Developer IP only  |
| 8081 | Adminer          | Developer IP only  |

Port 3306 (MySQL) is never exposed publicly. All application queries go through stored procedures called by the backend API, which connects internally via the Docker network.

## 6. PROJECT CONCLUSIONS AND RESULTS

### 6.1 SUMMARY OF DELIVERABLES

| Deliverable          | Status   | Details                                                                             |
|----------------------|----------|-------------------------------------------------------------------------------------|
| Conceptual Design    | Complete | Problem analysis, stakeholders, business rules, use-case/activity/sequence diagrams |
| ER Diagram           | Complete | 8 entities, all relationships with cardinalities defined                            |
| Relational Schema    | Complete | 8 normalized tables, all PKs and FKs defined                                        |
| Normalization        | Complete | 1NF, 2NF, 3NF verified for all tables                                               |
| Relational Algebra   | Complete | 18+ queries with algebra notation and SQL                                           |
| DDL (01_schema.sql)  | Complete | 8 tables, constraints, ENUMs, CHECK constraints                                     |
| DML (02_data.sql)    | Complete | 50+ records per table using WITH RECURSIVE                                          |
| DQL (03_queries.sql) | Complete | 10 analytical queries covering all main use cases                                   |
| Views (04, 10)       | Complete | 10 views including web-facing views with RANK()                                     |
| Indexes (05)         | Complete | 10 indexes covering all major join and filter patterns                              |
| Triggers (06)        | Complete | 4 triggers automating pixel assignment, logging, validation                         |
| Procedures (07, 09)  | Complete | 11 procedures including auth, canvas, leaderboard                                   |
| DCL (08)             | Complete | 2 DB users with role-based privileges                                               |
| Deployment           | Complete | Docker Compose on AWS EC2 t3.small                                                  |
| Frontend (optional)  | Complete | React + Vite app served via nginx in Docker                                         |

## 6.2 KEY DESIGN DECISIONS

- WITH RECURSIVE was used for data seeding instead of repetitive INSERT blocks, producing compact and maintainable seed scripts.
- Trigger logic handles pixel assignment automatically, removing the need for the application layer to manage this business rule explicitly.
- The NULL-safe comparison operator (<=>) in the pixel update trigger correctly detects ownership changes involving NULL values, which standard <> would miss.
- Subquery-based aggregation in views prevents count inflation that occurs when joining multiple one-to-many relationships directly.
- Submission timestamps are spaced one minute apart during seeding to avoid triggering the 5-second anti-spam rule.
- The app\_user database account is intentionally denied DELETE privileges; pixel releases use UPDATE to set owner to NULL instead.

## 6.3 POSSIBLE FUTURE IMPROVEMENTS

- Add a backend API layer (Node.js/Express) to connect the React frontend to MySQL through stored procedures.
- Implement JWT-based authentication instead of localStorage token storage.
- Add a real code execution engine to evaluate user submissions automatically rather than using seeded results.
- Add Prometheus and Grafana monitoring to track query performance and submission rates in production.
- Introduce database partitioning on the submissions table as data grows beyond millions of rows.

## CONCLUSION

The Coding Challenge and Pixel Ownership Platform has been successfully designed and implemented as a fully normalized relational database system. The project covers all required phases: conceptual analysis, logical design with ER modeling and normalization, and physical implementation in MySQL.

The database enforces all business rules through constraints, triggers, and stored procedures, ensuring data integrity without depending on application-layer validation. The system is deployed on AWS EC2 using Docker Compose and is ready to support a full-stack web application.